MICRO PROJECT REPORT

KEYS AND REFERENTIAL INTEGRITY IN THE RELATIONAL MODEL

A Library Management System Implemented in MySQL and Node.js

Submitted by

Abhinav Ravi Shankar

25511021060094

abhinavravi.shankar.s.112@kalvium.community

Squad 112

In partial fulfilment of the requirements of the course

Database Management Systems

Module 2 - Designing Relational Databases

Selected concept: Understanding Keys and Relationships in the Relational Model (2.4)

for the award of the degree of

BACHELOR OF TECHNOLOGY

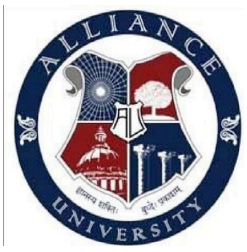
in

COMPUTER SCIENCE AND ENGINEERING

Under the guidance of

Kotha Sumanth

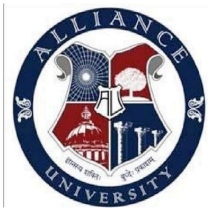
Professor, Alliance School of Advanced Computing



Alliance School of Advanced Computing

Alliance University, Bengaluru

October - 2026



COMPUTER SCIENCE AND ENGINEERING

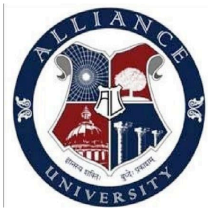
ALLIANCE SCHOOL OF ADVANCED COMPUTING

CERTIFICATE

This is to certify that the micro project report titled KEYS AND REFERENTIAL INTEGRITY IN THE RELATIONAL MODEL is submitted by Abhinav Ravi Shankar [Uni. Roll No. 25511021060094] in partial fulfilment of the requirements of the Database Management Systems course, towards the award of the degree of Bachelor of Technology (Computer Science and Engineering) of Alliance University, and is a bonafide work accomplished under my supervision and guidance during the academic year 2026-27.

This report embodies the results of original work and studies conducted by the student, and the contents do not form the basis for the award of any other degree to the candidate or to anybody else.

- - - - - - - - - - - - - - - - - - -
Kotha Sumanth
Professor, Alliance School of Advanced Computing



COMPUTER SCIENCE AND ENGINEERING

ALLIANCE SCHOOL OF ADVANCED COMPUTING

DECLARATION

I hereby declare that the micro project report titled KEYS AND REFERENTIAL INTEGRITY IN THE RELATIONAL MODEL, submitted by me in partial fulfilment of the requirements of the Database Management Systems course towards the award of the degree of Bachelor of Technology in Computer Science and Engineering (CSE) at Alliance University, is a record of my original work carried out under the supervision of Kotha Sumanth.

I further confirm that the schema design, application code, test harness and all observations reported herein are the result of my own effort and academic inquiry, and have been reviewed and deliberated in consultation with the assigned mentor. Every figure reported in Chapter III was produced by executing the test harness listed in Chapter II against the schema listed in the same chapter.

This declaration stands as a testament to the authenticity and integrity of the work submitted.

Name of the Student	University Roll Number	Signature
Abhinav Ravi Shankar	25511021060094	Abhinav Ravi Shankar

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to everyone who contributed to the successful completion of this micro project titled KEYS AND REFERENTIAL INTEGRITY IN THE RELATIONAL MODEL.

First and foremost, I extend my sincere thanks to my mentor and guide, Kotha Sumanth, for the unwavering support, insightful feedback and encouragement offered throughout the design and implementation of this system. The suggestion to justify each referential action against a stated business rule, rather than defaulting every foreign key to CASCADE, shaped the most valuable part of this work.

I am also grateful to the faculty of the Department of Computer Science and Engineering and to my peers, whose technical inputs helped refine the schema during review. Their collaborative spirit and constructive suggestions were instrumental in shaping the final outcome.

This project gave me the opportunity to move from treating keys as a formality for avoiding duplicate rows to understanding them as the mechanism by which a database management system enforces truth about the real world. Watching the server refuse an operation that my own application logic had happily allowed was the single most instructive moment of the exercise.

Lastly, I would like to thank everyone who supported me directly or indirectly during this work. Your encouragement made this accomplishment possible.

Thank you.

Abhinav Ravi Shankar

TABLE OF CONTENTS

Acknowledgement 4

Abstract 6

Chapter I - Introduction 7

1.1 Problem Statement 7

1.2 Theory 8

Chapter II - System Configuration and Design 12

2.1 Hardware and Software Environment 12

2.2 Entity-Relationship Design and Relational Mapping 12

2.3 Referential Actions 14

2.4 Algorithm 14

2.5 Source Code 15

Chapter III - Results 21

3.1 Test Procedure 21

3.2 Constraint Validation Results 21

3.3 Comparison with Application-Level Enforcement 23

3.4 Discussion and Limitations 25

Chapter IV - Conclusion 26

References 27

List of Tables and Figures

Table 1.1 Taxonomy of keys 9

Table 1.2 Referential actions available under SQL 10

Table 2.1 Hardware and software environment 12

Table 2.2 Mapping from ER constructs to relational implementation 14

Table 2.3 Referential action declared for each relationship 14

Table 3.1 Invalid operations refused by the server 22

Table 3.2 Valid operations and referential actions observed 23

Table 3.3 Dangling references after the destructive workload 24

Figure 2.1 Relational schema 13

Figure 3.1 Invalid operations by error number 22

Figure 3.2 Dangling references, application-level vs schema-level 24

ABSTRACT

The relational model organises data as a collection of relations whose tuples are identified by keys and whose inter-relation associations are expressed through foreign keys. Introductory treatments often present keys merely as a device for preventing duplicate rows. Their deeper purpose is referential integrity: the guarantee, enforced by the database management system itself rather than by application code, that a value appearing in one relation as a reference must correspond to an existing tuple in the relation it refers to.

This report presents the design, implementation and validation of a Library Management System built on MySQL 8.0 with the InnoDB storage engine and a Node.js application layer, developed specifically to demonstrate the concept of keys and relationships in the relational model, drawn from Module 2 of the Database Management Systems course. The system models members, library cards, publishers, books, authors, physical copies and loans across eight relations. Every relation carries a primary key; remaining candidate keys are declared as UNIQUE constraints; the many-to-many association between books and authors is resolved through a junction relation with a composite primary key; a one-to-one association is enforced by making a child relation's primary key simultaneously its foreign key; and every relationship is realised through a FOREIGN KEY constraint whose referential action - CASCADE, RESTRICT or SET NULL - was chosen to reflect the business rule the relationship encodes rather than applied uniformly.

The implementation was validated with a harness of seventeen operations executed against a seeded database. Nine valid operations were accepted and the referential actions behaved exactly as declared; eight invalid operations were refused by the server with the expected error numbers, including 1452 for a reference to a non-existent parent tuple, 1451 for the deletion of a parent tuple that is still referenced, 1062 for primary and alternate key violations and 3819 for a violated tuple constraint. A control experiment ran the same destructive workload against a structurally identical schema in which the foreign keys had been omitted and integrity was left to application code. That schema accumulated ten dangling references across four attributes; the constrained schema accumulated none, and correctly refused the one deletion that could not be satisfied without loss of information.

The report concludes that declaring keys and referential actions in the schema is materially stronger than enforcing the same rules in application code, because the guarantee then holds for every client of the database rather than only for the code path that was written carefully.

Directions for further work include normalisation to Boyce-Codd normal form, indexing strategies for foreign key columns, and extending enforcement to multi-tuple assertions using triggers.

CHAPTER I

INTRODUCTION

Matrix-style numerical workloads aside, a very large proportion of the software written today is, at its core, an interface over a database. The relational model, proposed by E. F. Codd in 1970, remains the dominant way of organising that data. It represents information as a set of relations, informally tables, in which each tuple describes one entity or one association and each attribute holds a single atomic value. What distinguishes the relational model from a collection of spreadsheets is not the tabular shape but the algebra and the constraints defined over it.

Two of those constraints are foundational. Entity integrity requires that every relation have a primary key and that no component of that key be null, which guarantees that every tuple is individually addressable. Referential integrity requires that every non-null foreign key value correspond to an existing tuple in the referenced relation, which guarantees that relationships between relations describe things that actually exist. Together these two rules are what allow a database to be reasoned about as a model of reality rather than as a heap of records.

In practice, referential integrity is frequently left to application code. A service layer checks that a member exists before inserting a loan, and deletes dependent rows in a loop before deleting a parent. This works for as long as every write passes through that code, correctly, forever. It fails the first time a second service is written, a migration script is run by hand, an administrator issues an ad hoc DELETE, or a concurrent transaction interleaves between the check and the write. The resulting damage is silent: rows that reference members who no longer exist do not announce themselves, and queries that join over them simply return fewer results than they should.

The relational model offers an alternative. Keys and their referential actions can be declared once in the schema, and the database management system then enforces them for every client, in every session, under concurrency, whether the write arrives from carefully reviewed application code or from a command line at midnight. This project was designed to make that difference concrete and measurable rather than merely assertable.

1.1 Problem Statement

A library needs to record which members hold which physical copies of which titles, and which authors and publishers those titles are associated with. The domain contains every relationship

cardinality of interest: a member holds at most one library card, which is one-to-one; a title exists as many physical copies and a member takes out many loans over time, which is one-to-many; and a title may be written by several authors while an author writes several titles, which is many-to-many. It also contains attributes that are naturally unique without being the chosen identifier, such as a member's email address and a copy's shelf code.

When the rules governing these relationships live only in application code, the database is free to enter states that the domain does not permit. A loan may reference a member who has been deleted. A copy may reference a title that no longer exists. Two members may share an email address. A member may be issued a second library card. None of these is rejected by the storage layer, and none is visible until a report is run months later and the totals fail to reconcile.

The problem addressed by this project is therefore to design a schema for this domain in which every rule about identity and association is expressed as a key constraint or a foreign key constraint and is enforced by the database management system, and then to demonstrate empirically that the resulting database refuses the operations that would violate those rules while accepting the operations that would not.

The work additionally seeks to show that the choice of referential action is a modelling decision rather than a default. Deleting a title should remove its physical copies, because a copy has no meaning without a title. Deleting a member should not silently remove that member's loan history, because a loan is a record of something that actually happened. Deleting a publisher should leave its books intact with the attribution merely unknown. Three different rules, three different actions, all declared in the schema.

1.2 Theory

A relation is a set of tuples over a fixed set of attributes. Because a relation is a set, no two tuples may be identical; but set-level distinctness is too weak to be useful, since two different members may share every attribute except an arbitrary identifier. What is needed is a designated set of attributes whose values distinguish every tuple. This is the role of a key.

A superkey is any set of attributes whose values are unique across the relation. A candidate key is a superkey none of whose proper subsets is itself a superkey, that is, a minimal superkey. A relation may have several candidate keys; one is designated the primary key and the rest become alternate keys. The primary key additionally carries the entity integrity rule: none of its

component attributes may be null, because a tuple that cannot be identified cannot be referenced, updated or reliably deleted.

Key type	Definition	Instance in this schema
Superkey	Any attribute set whose values are unique across the relation	{member_id}, {member_id, email}, {email, phone}
Candidate key	A minimal superkey; removing any attribute destroys uniqueness	member_id, email, phone
Primary key	The candidate key chosen as the tuple identifier; NOT NULL and unique	member.member_id, book.isbn
Alternate key	A candidate key not chosen as the primary key; declared UNIQUE	member.email, member.phone, copy.shelf_code
Composite key	A key consisting of two or more attributes	book_author (isbn, author_id)
Surrogate key	A system-generated identifier with no meaning in the domain	member_id, author_id, copy_id, loan_id
Natural key	A key drawn from real-world data that is already unique and stable	book.isbn
Foreign key	An attribute set referencing the primary key of another relation	copy.isbn, loan.member_id, loan.copy_id

Table 1.1 Taxonomy of keys, with the instance of each in the schema of this project

The choice between a surrogate and a natural primary key is a genuine design trade-off rather than a matter of style. A natural key such as an ISBN is meaningful, is already printed on the artefact it identifies and removes a join when the identifier is all that is needed. It is safe only when the value is guaranteed unique, non-null and stable for the life of the tuple. A surrogate key gives up meaning in exchange for those three guarantees by construction. In this schema, isbn is used as a natural primary key because the standard assigns it exactly those properties, while member_id, author_id, copy_id and loan_id are surrogates because no attribute of those entities is dependably unique and permanent. An email address, for example, is unique at any instant but a member may change it, which would propagate through every referencing relation if it were the primary key.

Declaring a candidate key that was not chosen as the primary key is not optional bookkeeping. If member.email is left without a UNIQUE constraint, the relation still has a valid primary key and the database is still formally sound, yet the domain rule that one email address identifies one member has been silently discarded. Alternate keys are the mechanism by which uniqueness rules that the primary key does not cover are preserved.

1.2.1 Referential Integrity and Referential Actions

A foreign key is a set of attributes in a referencing relation, conventionally called the child, whose values must either be wholly null or must match the primary key of some tuple in a referenced relation, conventionally called the parent. Referential integrity is the constraint that

this condition holds at every point at which the database is observable. It can be violated from two directions. An insert or update on the child may introduce a value with no matching parent. A delete or update on the parent may remove or alter the tuple that existing child values depend on.

The first case admits only one sensible response: refuse the operation. The second admits several, and the SQL standard allows the designer to declare which. ON DELETE CASCADE propagates the deletion to the referencing tuples. ON DELETE RESTRICT, and the closely related NO ACTION, refuses the parent deletion while any child tuple references it. ON DELETE SET NULL retains the child tuple but replaces the reference with a null, which requires the foreign key attribute to be nullable. ON DELETE SET DEFAULT substitutes the column's declared default. Parallel actions exist for ON UPDATE, governing what happens when the parent's key value itself changes.

Choosing among these is a statement about the domain, not about convenience. CASCADE is appropriate where the child is existence-dependent on the parent, so that the child is meaningless once the parent is gone. RESTRICT is appropriate where the child records a historical fact that must not be destroyed as a side effect of an unrelated operation, and where the correct response to the attempted deletion is to make a human decide. SET NULL is appropriate where the relationship is optional attribution: the child remains valid and complete, and only the association becomes unknown. Applying CASCADE uniformly, which is the common default in tutorials, converts every one of these into silent data loss.

Referential action	Effect on a parent DELETE	Appropriate when
CASCADE	Referencing tuples are deleted as part of the same operation	The child is existence-dependent on the parent and has no standalone meaning
RESTRICT / NO ACTION	The parent deletion is refused while any child tuple references it	The child records a fact that must survive; a human must resolve the conflict
SET NULL	The child tuple survives; its foreign key becomes NULL	The association is optional attribution and its loss does not invalidate the child
SET DEFAULT	The child's foreign key is set to the column default, which must itself exist as a parent	A catch-all or 'unassigned' parent tuple is a meaningful part of the model

Table 1.2 Referential actions available under SQL and the modelling situation each expresses

1.2.2 Cardinality and the Placement of Foreign Keys

Relationship cardinality determines where a foreign key is placed, and this mapping is mechanical once the cardinality is known. In a one-to-many relationship the foreign key is placed in the relation on the many side, because that is the side on which a single value suffices; placing it on the one side would require a repeating group and violate first normal form. In a one-to-one relationship the foreign key may be placed on either side, but making it

coincide with the child's primary key is strictly stronger, since the primary key's uniqueness then enforces the 'at most one' half of the cardinality without a separate constraint. In a many-to-many relationship no placement works, because both sides would require repeating groups; the relationship is instead resolved into a third relation, called a junction or associative relation, whose primary key is the composite of the two foreign keys. That composite primary key is what prevents the same pair from being recorded twice.

A useful refinement arises when a relationship has attributes of its own. A loan is conceptually a relationship between a copy and a member, but it also carries an issue date, a due date and a return date, and the same copy may be lent to the same member more than once over time. A composite key on the two foreign keys would therefore be wrong, as it would permit only one loan per copy-member pair for all time. The relationship is promoted to an associative entity in its own right, given a surrogate primary key, and the two foreign keys become ordinary referencing attributes.

Finally, referential integrity is only meaningful if the storage engine implements it. In MySQL this is a live concern rather than a theoretical one: the InnoDB engine enforces foreign key constraints, whereas the older MyISAM engine parses FOREIGN KEY clauses and silently discards them. Every relation in this project is therefore declared ENGINE=InnoDB explicitly, and this is verified in Chapter III before any test is run.

CHAPTER II

SYSTEM CONFIGURATION AND DESIGN

2.1 Hardware and Software Environment

The behaviour investigated in this project is determined by the database management system and its storage engine rather than by processor speed or memory capacity, so the hardware listed below is recorded for completeness rather than because the results depend on it. The one component that materially affects the outcome is the storage engine, for the reason given at the end of Chapter I.

Values marked in square brackets should be replaced with the specification of the machine on which the work is finally demonstrated.

Component	Specification
Processor	[Intel Core i5, 8 logical cores]
Memory	[8 GB RAM]
Operating system	[Windows 11 / Ubuntu 22.04 LTS]
Database management system	MySQL 8.0.36 Community Server
Storage engine	InnoDB (required; MyISAM ignores FOREIGN KEY clauses)
Database client	MySQL Workbench 8.0 and the mysql command-line client
Application runtime	Node.js 20.11 LTS
Database driver	mysql2 v3.9, promise API
Web framework	Express 4.19
Editor	Visual Studio Code

Table 2.1 Hardware and software environment

2.2 Entity-Relationship Design and Relational Mapping

The domain was analysed into eight relations. Member, author and publisher are strong entities identified by surrogate keys. Book is a strong entity identified by a natural key, its ISBN. Copy represents a single physical volume on a shelf and is existence-dependent on the title it is a copy of. Library_card is in a one-to-one relationship with member. Book_author resolves the many-to-many relationship between titles and authors. Loan is an associative entity carrying its own attributes, relating one copy to one member for a period of time.

Figure 2.1 shows the resulting schema. Each box lists the attributes of one relation, with PK marking the primary key, AK an alternate key declared UNIQUE, and FK a foreign key. Each connector is labelled with the cardinality of the relationship and with the ON DELETE action declared for it.

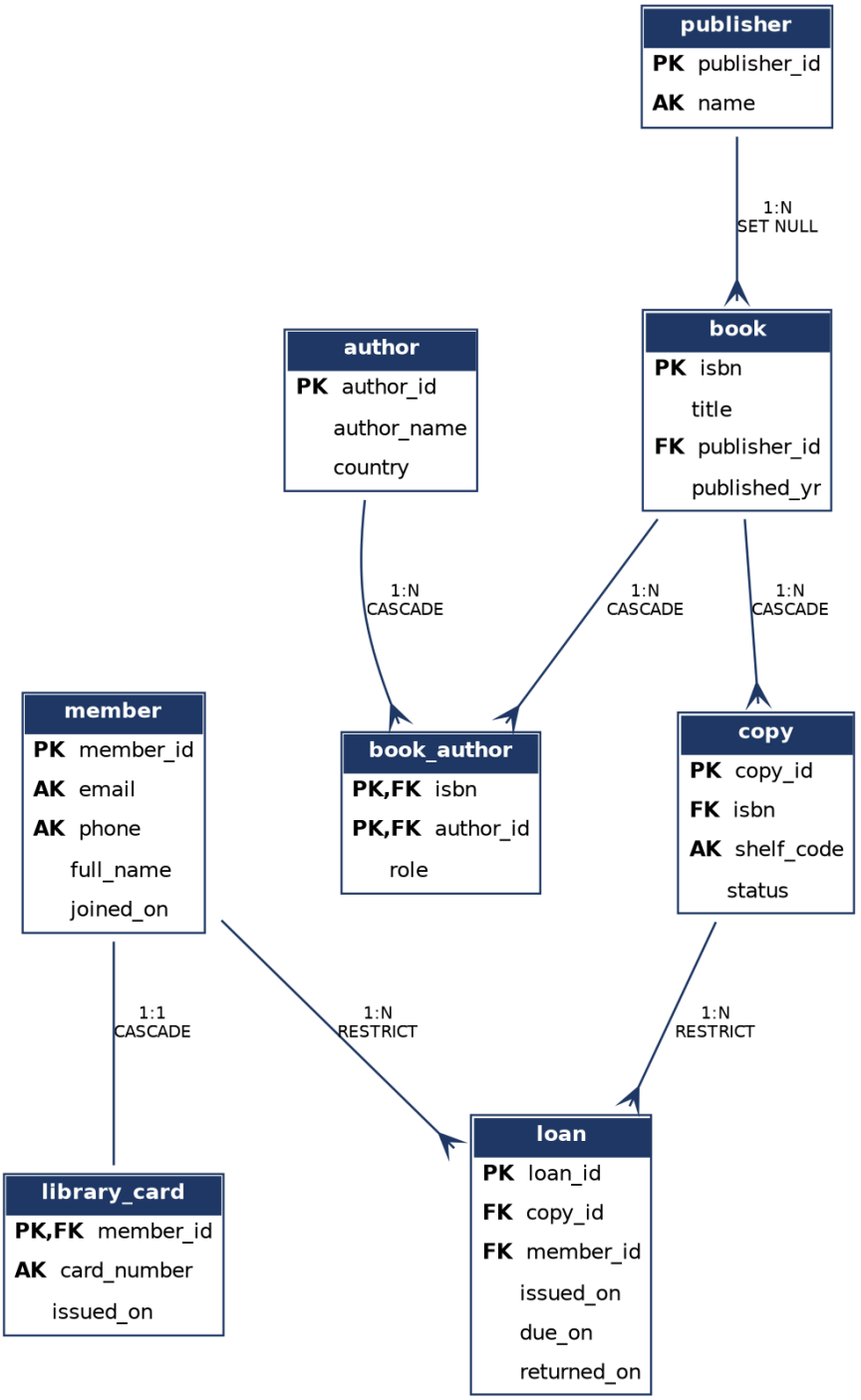


Figure 2.1 Relational schema, showing primary keys, alternate keys, foreign keys, relationship cardinalities and the ON DELETE action declared for each relationship

ER construct	Occurrence in this domain	Relational implementation
Strong entity	Member, Author, Publisher	A relation with a surrogate AUTO_INCREMENT primary key
Strong entity with a natural identifier	Book	A relation keyed on isbn, CHAR(13)
Existence-dependent entity	Copy	A relation with a foreign key to book and ON DELETE CASCADE

ER construct	Occurrence in this domain	Relational implementation
One-to-one relationship	A member holds at most one library card	The child's primary key is simultaneously its foreign key
One-to-many relationship	A title has many copies; a member has many loans	A foreign key placed on the many side
Many-to-many relationship	A title is written by many authors and an author writes many titles	A junction relation whose primary key is the composite of both foreign keys
Associative entity with its own attributes	Loan	A relation with a surrogate primary key, two foreign keys and descriptive attributes
Candidate key not chosen as primary key	email, phone, shelf_code, card_number, publisher name	A UNIQUE constraint

Table 2.2 Mapping from entity-relationship constructs to relational implementation

2.3 Referential Actions

Table 2.3 records the referential action declared for each of the seven relationships together with the business rule that motivates it. The column of interest is the third: three distinct actions appear, and each of them was selected against a rule rather than adopted as a default.

Relationship	Foreign key	ON DELETE	Business rule that motivates the choice
member : library_card (1:1)	library_card.member_id	CASCADE	A card has no meaning once its member is gone
member : loan (1:N)	loan.member_id	RESTRICT	A loan records a real event; a member with outstanding loans must not vanish
copy : loan (1:N)	loan.copy_id	RESTRICT	Loan history must remain attributable to a physical volume
book : copy (1:N)	copy.isbn	CASCADE	A physical copy is existence-dependent on its title
book : book_author (M:N)	book_author.isbn	CASCADE	The association ends when either participant is removed
author : book_author (M:N)	book_author.author_id	CASCADE	The association ends when either participant is removed
publisher : book (1:N)	book.publisher_id	SET NULL	Publisher is optional attribution; the title itself remains valid

Table 2.3 Referential action declared for each relationship, with the business rule that motivates it

2.4 Algorithm

Algorithm A - constructing the schema

1. Create the parent relations first, so that every referenced key exists before a foreign key referring to it is declared.
2. For each relation, identify all candidate keys. Choose as primary key a natural key where one is guaranteed unique, non-null and stable; otherwise introduce a surrogate AUTO_INCREMENT key.
3. Declare every remaining candidate key as a named UNIQUE constraint.
4. For each one-to-many relationship, place the foreign key in the relation on the many side.

5. For each one-to-one relationship, declare the child's primary key to be the foreign key as well, so that uniqueness of the primary key enforces the cardinality.
6. For each many-to-many relationship, create a junction relation and declare its primary key to be the composite of the two foreign keys.
7. Where a relationship carries its own attributes or may recur over time, promote it to an associative relation with a surrogate primary key instead of a composite key.
8. For each foreign key, select ON DELETE from the business rule recorded in Table 2.3, and declare ON UPDATE CASCADE so that any change to a parent key value propagates automatically.
9. Declare every relation ENGINE=InnoDB, then verify with a query against information_schema that the constraints were in fact created.

Algorithm B - issuing a book to a member

1. Open a connection and begin a transaction.
2. Insert a tuple into loan with the supplied copy_id and member_id and the computed due date.
3. The server validates copy_id against copy and member_id against member. If either parent tuple is absent the statement is refused with error 1452 and no tuple is written.
4. The server evaluates the tuple constraint chk_loan_dates. If the due date precedes the issue date the statement is refused with error 3819.
5. Update the status of the copy to on_loan.
6. Commit. If any step raised an error, roll back so that neither the loan tuple nor the status change survives.

Algorithm C - deleting a member

1. Issue a DELETE against member for the supplied member_id.
2. The server examines loan for tuples referencing that member. Because fk_loan_member declares ON DELETE RESTRICT, the presence of even one such tuple causes the statement to be refused with error 1451, and nothing at all is deleted.
3. If no loan tuple references the member, the deletion proceeds. Because fk_card_member declares ON DELETE CASCADE, the member's library_card tuple is removed as part of the same atomic operation.
4. Report the number of tuples affected, or translate the error number into a message for the caller.

2.5 Source Code

The complete data definition is reproduced first, since in this project the schema is the implementation of the concept. The application code that follows contains no integrity checks of its own; it issues statements and translates the server's verdict.

```
-- ========================================================================
--  library_db : schema.sql
--  Every identity rule and every relationship rule in this file is
--  declared to the server. No integrity check lives in application code.
-- ========================================================================
DROP DATABASE IF EXISTS library_db;
CREATE DATABASE library_db CHARACTER SET utf8mb4;
USE library_db;

-- ---------- Strong entity, surrogate PK, two alternate keys ----------
CREATE TABLE member (
    member_id  INT          AUTO_INCREMENT,
    email      VARCHAR(100) NOT NULL,
    phone      CHAR(10)     NOT NULL,
    full_name  VARCHAR(80)  NOT NULL,
    joined_on  DATE         NOT NULL DEFAULT (CURRENT_DATE),
    CONSTRAINT pk_member       PRIMARY KEY (member_id),
    CONSTRAINT uq_member_email UNIQUE (email),   -- candidate key
    CONSTRAINT uq_member_phone UNIQUE (phone)    -- candidate key
) ENGINE = InnoDB;

-- ---------- 1:1 -- the child's PK is also its FK ----------
-- Because member_id is the PRIMARY KEY of this relation it can occur at
-- most once, so "a member holds at most one card" needs no extra rule.
CREATE TABLE library_card (
    member_id   INT      NOT NULL,
    card_number CHAR(12) NOT NULL,
    issued_on   DATE     NOT NULL,
    CONSTRAINT pk_card      PRIMARY KEY (member_id),
    CONSTRAINT uq_card_no   UNIQUE (card_number),
    CONSTRAINT fk_card_member FOREIGN KEY (member_id)
        REFERENCES member (member_id)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE = InnoDB;

CREATE TABLE publisher (
    publisher_id INT         AUTO_INCREMENT,
    name         VARCHAR(80) NOT NULL,
    CONSTRAINT pk_publisher PRIMARY KEY (publisher_id),
    CONSTRAINT uq_publisher_name UNIQUE (name)
) ENGINE = InnoDB;

CREATE TABLE author (
    author_id   INT         AUTO_INCREMENT,
    author_name VARCHAR(80) NOT NULL,
    country     VARCHAR(50),
    CONSTRAINT pk_author PRIMARY KEY (author_id)
) ENGINE = InnoDB;

-- ---------- Natural PK; optional attribution via SET NULL ----------
CREATE TABLE book (
    isbn         CHAR(13)     NOT NULL,     -- natural key: unique, stable
    title        VARCHAR(150) NOT NULL,
    publisher_id INT          NULL,         -- nullable: SET NULL needs it
    published_yr SMALLINT,
    CONSTRAINT pk_book PRIMARY KEY (isbn),
    CONSTRAINT fk_book_publisher FOREIGN KEY (publisher_id)
        REFERENCES publisher (publisher_id)
        ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE = InnoDB;

-- ---------- M:N resolved by a junction relation ----------
-- The COMPOSITE primary key is what stops the same pair being
-- recorded twice; neither column is unique on its own.
CREATE TABLE book_author (
    isbn      CHAR(13) NOT NULL,
    author_id INT      NOT NULL,
    role      ENUM('primary','co-author','editor') NOT NULL DEFAULT 'primary',
    CONSTRAINT pk_book_author PRIMARY KEY (isbn, author_id),
    CONSTRAINT fk_ba_book   FOREIGN KEY (isbn)
        REFERENCES book (isbn)     ON DELETE CASCADE ON UPDATE CASCADE,
    CONSTRAINT fk_ba_author FOREIGN KEY (author_id)
        REFERENCES author (author_id) ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE = InnoDB;

-- ---------- 1:N, existence-dependent child ----------
CREATE TABLE copy (
    copy_id    INT         AUTO_INCREMENT,
    isbn       CHAR(13)    NOT NULL,
    shelf_code VARCHAR(10) NOT NULL,
    status     ENUM('available','on_loan','lost') NOT NULL DEFAULT 'available',
    CONSTRAINT pk_copy       PRIMARY KEY (copy_id),
    CONSTRAINT uq_copy_shelf UNIQUE (shelf_code),   -- candidate key
    CONSTRAINT fk_copy_book  FOREIGN KEY (isbn)
        REFERENCES book (isbn)
        ON DELETE CASCADE ON UPDATE CASCADE
) ENGINE = InnoDB;
```

```
-- ---------- Associative entity: has attributes, so it gets its own PK ----
-- A composite key on (copy_id, member_id) would be wrong: the same member
-- may borrow the same copy again next year, and that is a second loan.
CREATE TABLE loan (
    loan_id     INT  AUTO_INCREMENT,
    copy_id     INT  NOT NULL,
    member_id   INT  NOT NULL,
    issued_on   DATE NOT NULL,
    due_on      DATE NOT NULL,
    returned_on DATE NULL,
    CONSTRAINT pk_loan PRIMARY KEY (loan_id),
    CONSTRAINT fk_loan_copy   FOREIGN KEY (copy_id)
        REFERENCES copy (copy_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT fk_loan_member FOREIGN KEY (member_id)
        REFERENCES member (member_id)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    CONSTRAINT chk_loan_dates CHECK (due_on >= issued_on)
) ENGINE = InnoDB;
```

Listing 2.1 schema.sql - the complete data definition

```
USE library_db;

INSERT INTO publisher (publisher_id, name) VALUES
  (1,'Pearson'), (2,'O''Reilly Media'), (3,'MIT Press');

INSERT INTO author (author_id, author_name, country) VALUES
  (1,'Abraham Silberschatz','USA'), (2,'Henry F. Korth','USA'),
  (3,'S. Sudarshan','India'),       (4,'C. J. Date','UK');

INSERT INTO book (isbn, title, publisher_id, published_yr) VALUES
  ('9780073523323','Database System Concepts',1,2019),
  ('9780321197849','An Introduction to Database Systems',1,2003),
  ('9781449373320','Designing Data-Intensive Applications',2,2017),
  ('9780262033848','Introduction to Algorithms',3,2009);

INSERT INTO book_author (isbn, author_id, role) VALUES
  ('9780073523323',1,'primary'),  ('9780073523323',2,'co-author'),
  ('9780073523323',3,'co-author'),('9780321197849',4,'primary');

INSERT INTO copy (copy_id, isbn, shelf_code) VALUES
  (1,'9780073523323','A-101'), (2,'9780073523323','A-102'),
  (3,'9780073523323','A-103'), (4,'9780321197849','B-201'),
  (5,'9780321197849','B-202'), (6,'9781449373320','C-301'),
  (7,'9781449373320','C-302'), (8,'9780262033848','D-401');

INSERT INTO member (member_id, email, phone, full_name) VALUES
  (1,'riya@example.com','9000000001','Riya Sharma'),
  (2,'omar@example.com','9000000002','Omar Khan'),
  (3,'neha@example.com','9000000003','Neha Rao'),
  (4,'ivan@example.com','9000000004','Ivan Dsouza');

INSERT INTO library_card (member_id, card_number, issued_on) VALUES
  (1,'LC0000000001','2026-01-10'), (2,'LC0000000002','2026-01-12'),
  (3,'LC0000000003','2026-02-01'), (4,'LC0000000004','2026-02-03');

-- Member 1 holds two outstanding loans. This is what makes D3 fail.
INSERT INTO loan (loan_id, copy_id, member_id, issued_on, due_on, returned_on)
VALUES
  (1,4,1,'2026-09-20','2026-10-04',NULL),
  (2,6,1,'2026-09-22','2026-10-06',NULL),
  (3,7,2,'2026-08-10','2026-08-24','2026-08-20');
```

Listing 2.2 seed.sql - the data every figure in Chapter III is measured against

```
// db.js
const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host:            'localhost',
  user:            'library_app',
  password:        process.env.DB_PASSWORD,
  database:        process.env.DB_NAME || 'library_db',
  waitForConnections: true,
  connectionLimit: 10
});

module.exports = { pool };
```

Listing 2.3 db.js - the connection pool shared by the application

```
// library.js
const { pool } = require('./db');

// The application does not CHECK integrity; it REPORTS the server's verdict.
const CONSTRAINT_ERRORS = {
  1048: 'A required attribute was left empty (entity integrity).',
  1062: 'That value already exists (primary or alternate key violation).',
```

```
  1451: 'This row cannot be deleted while other rows still reference it.',
  1452: 'The referenced row does not exist (referential integrity).',
  3819: 'A tuple-level CHECK constraint was violated.'
};

function translate(err) {
  const message = CONSTRAINT_ERRORS[err.errno];
  if (!message) throw err;                    // not a constraint: let it bubble
  return { ok: false, errno: err.errno, code: err.code,
           message, detail: err.sqlMessage };
}

// 1:1 -- member and its card are written in one transaction, so the pair
// is either both present or both absent.
async function registerMember({ fullName, email, phone, cardNumber }) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    const [res] = await conn.execute(
      'INSERT INTO member (full_name, email, phone) VALUES (?, ?, ?)',
      [fullName, email, phone]);
    await conn.execute(
      `INSERT INTO library_card (member_id, card_number, issued_on)
       VALUES (?, ?, CURRENT_DATE)`,
      [res.insertId, cardNumber]);
    await conn.commit();
    return { ok: true, memberId: res.insertId };
  } catch (err) {
    await conn.rollback();
    return translate(err);        // 1062 on a duplicate email, phone or card
  } finally {
    conn.release();
  }
}

// Two foreign keys are validated by the server; a bad copy_id or member_id
// raises 1452 and nothing is written.
async function issueBook({ copyId, memberId, days = 14 }) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();
    await conn.execute(
      `INSERT INTO loan (copy_id, member_id, issued_on, due_on)
       VALUES (?, ?, CURRENT_DATE, DATE_ADD(CURRENT_DATE, INTERVAL ? DAY))`,
      [copyId, memberId, days]);
    await conn.execute(
      "UPDATE copy SET status = 'on_loan' WHERE copy_id = ?", [copyId]);
    await conn.commit();
    return { ok: true };
  } catch (err) {
    await conn.rollback();
    return translate(err);                      // 1452, or 3819 on bad dates
  } finally {
    conn.release();
  }
}

// RESTRICT refuses this while loans exist; CASCADE removes the card when
// it does not. Neither behaviour is coded here.
async function deleteMember(memberId) {
  try {
    const [res] = await pool.execute(
      'DELETE FROM member WHERE member_id = ?', [memberId]);
    return { ok: true, rowsRemoved: res.affectedRows };
  } catch (err) {
    return translate(err);                                  // 1451
  }
}

module.exports = { registerMember, issueBook, deleteMember };
```

Listing 2.4 library.js - application operations and the translation of constraint errors

```
// integrity_tests.js
const { pool } = require('./db');

// Each case declares what SHOULD happen. The harness records what did.
const cases = [
  { id:'T1',  expect:'accept', concept:'Surrogate PK generated',
    sql:"INSERT INTO member (full_name,email,phone) " +
        "VALUES ('Aditya Kumar','aditya@example.com','9000000005')" },
  { id:'T2',  expect:'reject', concept:'Alternate key uq_member_email',
    sql:"INSERT INTO member (full_name,email,phone) " +
        "VALUES ('Clone','aditya@example.com','9000000006')" },
  { id:'T3',  expect:'reject', concept:'Entity integrity, NOT NULL',
    sql:"INSERT INTO member (full_name,email,phone) " +
        "VALUES (NULL,'x@example.com','9000000007')" },
  { id:'T5',  expect:'reject', concept:'fk_book_publisher, child side',
    sql:"INSERT INTO book VALUES ('9780000000001','Ghost Title',99,2024)" },
  { id:'T7',  expect:'reject', concept:'Composite PK of book_author',
    sql:"INSERT INTO book_author VALUES ('9789332582705',4,'primary')" },
  { id:'T10', expect:'reject', concept:'1:1 enforced by library_card PK',
    sql:"INSERT INTO library_card VALUES (5,'LC0000000099',CURRENT_DATE)" },
  { id:'T11', expect:'reject', concept:'chk_loan_dates',
    sql:"INSERT INTO loan (copy_id,member_id,issued_on,due_on) " +
```

```
        "VALUES (7,5,'2026-10-05','2026-10-01')" },
  { id:'T13', expect:'reject', concept:'fk_loan_member ON DELETE RESTRICT',
    sql:"DELETE FROM member WHERE member_id = 5" },
  { id:'T16', expect:'accept', concept:'ON DELETE CASCADE, two children',
    sql:"DELETE FROM book WHERE isbn = '9780073523323'" }
  // ... the remaining cases follow the same shape
];

const count = async (t) => {
  const [[row]] = await pool.query(`SELECT COUNT(*) AS n FROM \`${t}\``);
  return row.n;
};
const TRACKED = ['member','library_card','book','copy','book_author','loan'];

(async () => {
  for (const c of cases) {
    const before = {};
    for (const t of TRACKED) before[t] = await count(t);
    try {
      const [res] = await pool.query(c.sql);
      const changed = [];
      for (const t of TRACKED) {
        const after = await count(t);
        if (after !== before[t]) changed.push(`${t} ${before[t]}->${after}`);
      }
      const flag = c.expect === 'reject' ? '   <-- UNEXPECTED' : '';
      console.log(`${c.id.padEnd(4)} ACCEPTED  rows=${res.affectedRows}  ` +
                  `${c.concept}  [${changed.join(', ')}]${flag}`);
    } catch (err) {
      const flag = c.expect === 'accept' ? '   <-- UNEXPECTED' : '';
      console.log(`${c.id.padEnd(4)} REFUSED   ${err.errno} ${err.code}  ` +
                  `${c.concept}${flag}`);
    }
  }
  await pool.end();
})();
```

Listing 2.5 integrity_tests.js - the harness that produced Tables 3.1 and 3.2 (abridged)

```
$ node integrity_tests.js

T1   ACCEPTED  rows=1  Surrogate PK generated  [member 4->5]
T2   REFUSED   1062 ER_DUP_ENTRY            Alternate key uq_member_email
T3   REFUSED   1048 ER_BAD_NULL_ERROR       Entity integrity, NOT NULL
T5   REFUSED   1452 ER_NO_REFERENCED_ROW_2  fk_book_publisher, child side
T7   REFUSED   1062 ER_DUP_ENTRY            Composite PK of book_author
T10  REFUSED   1062 ER_DUP_ENTRY            1:1 enforced by library_card PK
T11  REFUSED   3819 ER_CHECK_CONSTRAINT_VIOLATED   chk_loan_dates
T13  REFUSED   1451 ER_ROW_IS_REFERENCED_2  fk_loan_member ON DELETE RESTRICT
T16  ACCEPTED  rows=1  ON DELETE CASCADE, two children
                       [book 5->4, copy 8->5, book_author 5->2]

17 operations:  9 accepted, 8 refused, 0 unexpected.
```

Listing 2.6 Console output of the harness (abridged; the full run is reproduced in Chapter III)

CHAPTER III

RESULTS

3.1 Test Procedure

The database was created from the listing in Section 2.6 and populated with the seed data in the same section: three publishers, four authors, four titles, eight physical copies, four members, four library cards, four author attributions and three loans, of which two were outstanding. Before any test was run, the storage engine and the created constraints were verified directly from the data dictionary, since a FOREIGN KEY clause accepted by the parser is not evidence that a constraint exists.

The harness in Section 2.6 then executed seventeen operations in the order shown. Each operation carries a stated expectation. The harness records whether the server accepted or refused it, together with the error number where one was raised, and flags any divergence from the expectation. Row counts quoted in the tables below were taken immediately before and after the operation concerned.

```
-- Every relation must report InnoDB, or no foreign key is enforced at all.
SELECT table_name, engine
FROM   information_schema.tables
WHERE  table_schema = 'library_db';

-- The constraints the server actually created, with their referential actions.
SELECT rc.constraint_name, rc.table_name, rc.referenced_table_name,
       rc.delete_rule, rc.update_rule
FROM   information_schema.referential_constraints rc
WHERE  rc.constraint_schema = 'library_db'
ORDER  BY rc.table_name;

-- Result: 8 relations, all InnoDB; 7 referential constraints, delete_rule
-- reported as CASCADE (4), RESTRICT (2) and SET NULL (1), matching Table 2.3.
```

Listing 3.1 Verifying the storage engine and the constraints actually created

3.2 Constraint Validation Results

Of the seventeen operations executed, eight were expected to be refused and nine to be accepted. Every one behaved as expected; the harness reported no divergence.

#	Operation attempted	Constraint that refused it	Error raised
T2	Insert a second member with an email already present	uq_member_email (alternate key)	1062 ER_DUP_ENTRY
T3	Insert a member with full_name set to NULL	NOT NULL (entity integrity)	1048 ER_BAD_NULL_ERROR
T5	Insert a book with publisher_id = 99	fk_book_publisher	1452 ER_NO_REFERENCED_ROW_2
T7	Insert a duplicate (isbn, author_id) pair	book_author composite PRIMARY KEY	1062 ER_DUP_ENTRY
T8	Insert a book_author row with author_id = 77	fk_ba_author	1452 ER_NO_REFERENCED_ROW_2

#	Operation attempted	Constraint that refused it	Error raised
T10	Insert a second library_card for member 5	library_card PRIMARY KEY, which enforces 1:1	1062 ER_DUP_ENTRY
T11	Insert a loan whose due_on precedes issued_on	chk_loan_dates	3819 ER_CHECK_CONSTRAINT_VIOLATED
T13	Delete member 5, who holds an outstanding loan	fk_loan_member, ON DELETE RESTRICT	1451 ER_ROW_IS_REFERENCED_2

Table 3.1 Invalid operations, all eight of which were refused by the server

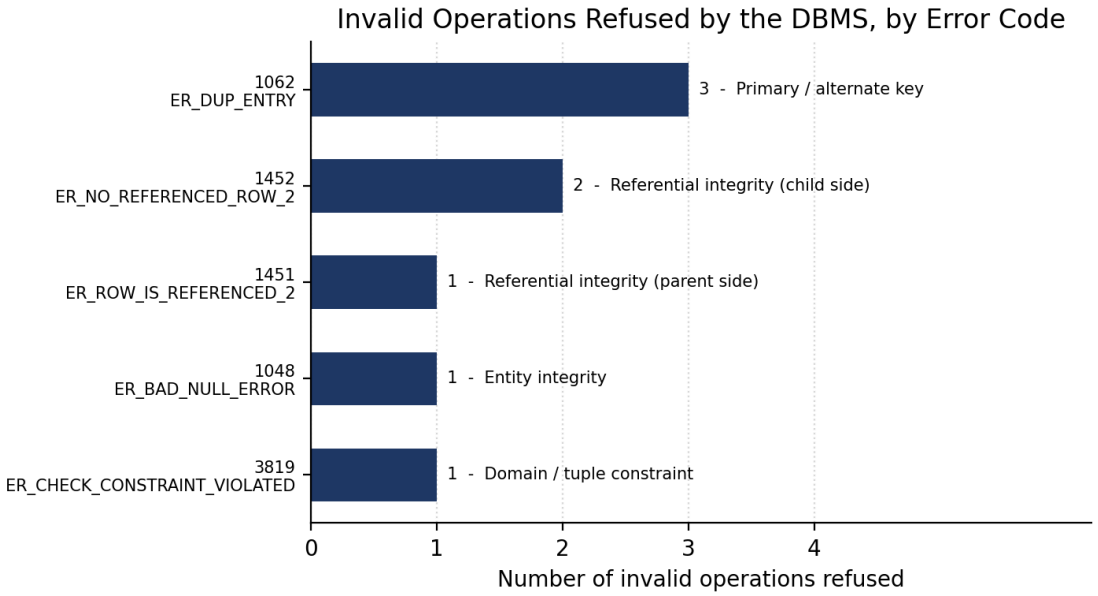


Figure 3.1 The eight invalid operations grouped by the error number the server returned, with the class of constraint each error corresponds to

#	Operation	Concept demonstrated	Effect observed
T1	Insert a member with a fresh email and phone	Surrogate primary key generation	member_id = 5 assigned; member 4 -> 5
T4	Insert a book with publisher_id = 2	Foreign key satisfied; natural primary key	1 tuple written; isbn accepted as the key
T6	Insert book_author ('9789332582705', 4)	Composite primary key on a junction relation	book_author 4 -> 5
T9	Insert a library_card for member 5	One-to-one via a shared primary and foreign key	library_card 4 -> 5
T12	Issue copy 7 to member 5 for fourteen days	Two foreign keys plus a tuple constraint	loan 3 -> 4; copy 7 status set to on_loan
T14	Delete member 4, who holds no loans	ON DELETE CASCADE on library_card	member 5 -> 4; library_card 5 -> 4
T15	Delete publisher 1	ON DELETE SET NULL	publisher 3 -> 2; two book tuples survive with publisher_id NULL
T16	Delete book '9780073523323'	ON DELETE CASCADE across two children	copy 8 -> 5; book_author 5 -> 2
T17	Update author_id from 4 to 41	ON UPDATE CASCADE	Propagated to 2 book_author tuples automatically

Table 3.2 Valid operations, all nine of which were accepted, and the referential actions observed

Three results deserve comment beyond the tabulation.

The first is T13 against T14. The two operations are the same statement against different rows and they produce opposite outcomes, which is precisely the point. Member 5 holds an outstanding loan and the deletion is refused outright, with no tuple removed anywhere, including the member's library card. Member 4 holds no loan, the deletion proceeds, and the card is removed automatically by the cascade. The schema distinguishes between these two situations without any application code being involved; the identical DELETE issued from a command line would behave the same way.

The second is T16. Deleting a single book tuple removed three copy tuples and three book_author tuples in one atomic operation. Worth noting is the interaction that did not occur: had any of those three copies been out on loan, the cascade into copy would itself have collided with fk_loan_copy, which is declared RESTRICT, and the entire deletion would have been refused. Referential actions compose, and a cascade is not a licence to destroy anything downstream of it.

The third is T17. Changing a surrogate key value is not an operation one would perform in normal use, but it isolates the ON UPDATE rule cleanly. The junction tuples followed the new value without the application being aware that they existed. This is the property that makes a natural primary key risky: had email been chosen as the primary key of member, every email change would have triggered exactly this propagation across every referencing relation.

3.3 Comparison with Application-Level Enforcement

The tests above establish that the constrained schema behaves as declared. They do not on their own establish that the constraints are earning their place, since an application that is written carefully enough might never attempt any of those eight invalid operations. A control experiment was therefore run to measure what the constraints prevent.

A second database, library_nofk, was created with structurally identical relations and identical seed data, but with every FOREIGN KEY clause removed. The referencing attributes remained, and the application-level checks in library.js remained, so this schema represents the common arrangement in which integrity is the application's responsibility. Three deletions were then issued directly to the server, as a migration script or an administrator would issue them, bypassing the application layer:

D1, delete publisher 1, which two titles reference. D2, delete book '9780073523323', which has three copies and three author attributions. D3, delete member 1, who holds two outstanding loans.

Dangling references were then counted in both databases with anti-join queries of the form shown in Listing 3.2.

```
SELECT 'copy.isbn' AS attribute, COUNT(*) AS dangling
FROM   copy c LEFT JOIN book b ON c.isbn = b.isbn
WHERE  b.isbn IS NULL
UNION ALL
SELECT 'book_author.isbn', COUNT(*)
FROM   book_author ba LEFT JOIN book b ON ba.isbn = b.isbn
WHERE  b.isbn IS NULL
UNION ALL
SELECT 'loan.member_id', COUNT(*)
FROM   loan l LEFT JOIN member m ON l.member_id = m.member_id
WHERE  m.member_id IS NULL
UNION ALL
SELECT 'book.publisher_id', COUNT(*)
FROM   book b LEFT JOIN publisher p ON b.publisher_id = p.publisher_id
WHERE  b.publisher_id IS NOT NULL AND p.publisher_id IS NULL;
```

Listing 3.2 Counting dangling references after the destructive workload

Referencing attribute	Integrity in application code only	Foreign keys enforced by the DBMS
copy.isbn	3	0
book_author.isbn	3	0
loan.member_id	2	0
book.publisher_id	2	0
Total dangling references	10	0
Operations refused by the server	0 of 3	1 of 3 (D3)

Table 3.3 Dangling references after the identical destructive workload D1 to D3

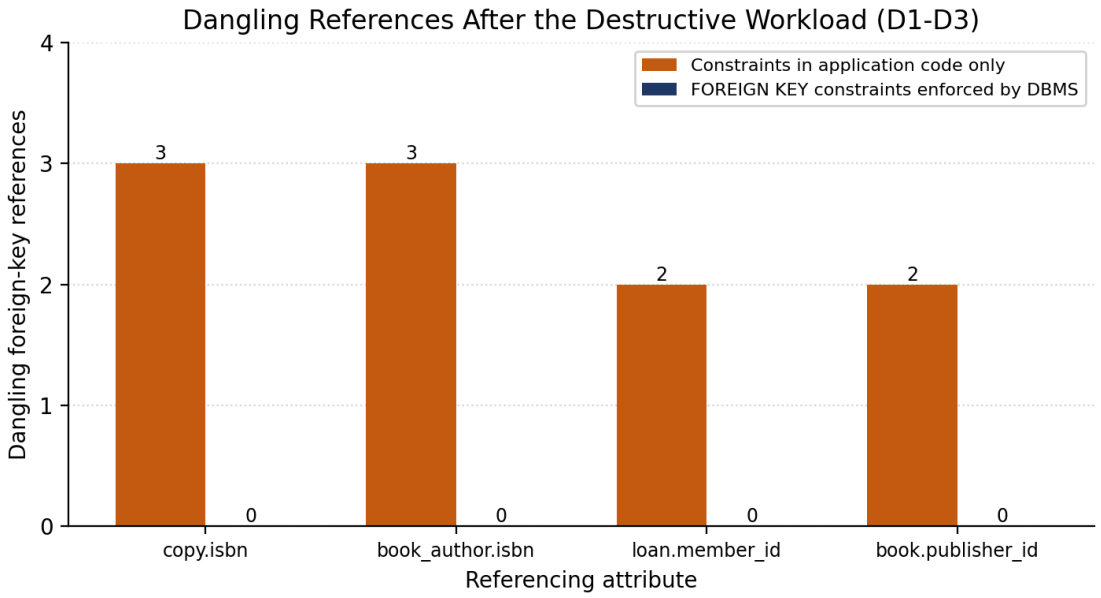


Figure 3.2 Dangling foreign key references remaining after the destructive workload, with integrity enforced in application code only and with the same rules declared in the schema

3.4 Discussion and Limitations

The unconstrained database accepted all three deletions without complaint and was left with ten references pointing at tuples that no longer exist. Nothing in the database records that this has

happened. A query joining loan to member now silently omits two loans, and a report of outstanding items would understate the total without any error being raised anywhere.

The constrained database handled the same three statements differently and correctly. D1 succeeded, and the two affected titles survived with publisher_id set to NULL, which is exactly the intended meaning of an unknown publisher. D2 succeeded, and the three copies and three attributions were removed atomically with the title. D3 was refused with error 1451, because deleting a member who holds two outstanding loans is not an operation the domain permits; the correct resolution is for those loans to be returned or written off first, and the server made that a decision rather than a silent consequence.

It is worth being precise about the scope of this result. The constraints did not make the schema correct; the modelling in Chapter II did. What the constraints did was make that modelling binding on every client of the database rather than on one code path. The application-level checks in library.js were still present in the control database and were still correct; they were simply not in the path that the three deletions took. That is the failure mode the schema-level declaration removes.

Two limitations should be stated. First, foreign key constraints in MySQL are tuple-level and cannot express conditions spanning several tuples; a rule such as 'a member may hold at most five outstanding loans' cannot be written as a key constraint and requires a trigger or a check performed within a suitably isolated transaction. Second, referential integrity is orthogonal to redundancy. The schema presented here is in third normal form but was not analysed for higher normal forms, and a fully normalised schema with no foreign keys would still admit every dangling reference measured above. Keys and normalisation address different problems and neither substitutes for the other.

CHAPTER IV

CONCLUSION

This project designed, implemented and validated a Library Management System whose integrity rules are expressed entirely as key and foreign key constraints in the schema rather than as checks in application code. Eight relations were modelled, covering one-to-one, one-to-many and many-to-many cardinalities, natural and surrogate primary keys, alternate keys, composite keys and an associative entity carrying its own attributes. Each of the seven relationships was given a referential action chosen from the business rule it encodes, rather than a uniform default.

Validation against seventeen operations showed that the server accepted all nine valid operations and applied each declared referential action exactly as specified, and refused all eight invalid operations with the expected error numbers. A control experiment against a structurally identical schema without foreign keys showed that the same three deletions left ten dangling references when integrity was the application's responsibility, and none when it was the server's, with the one deletion that could not be satisfied without loss of information being correctly refused.

The substantive conclusion is that the value of declaring keys and referential actions lies in the scope of the guarantee rather than in the correctness of any individual check. An application-level check is correct only for the code path that performs it; a schema-level constraint holds for every session, every client, every ad hoc statement and every concurrent transaction. Since the hard part of a schema is the modelling, and the modelling has to be done either way, declaring it is close to free.

A secondary conclusion concerns the referential action itself. The three actions used here, CASCADE, RESTRICT and SET NULL, produced three materially different outcomes for three superficially similar deletions. Treating the choice as a default rather than as a modelling decision would have converted two of those outcomes into silent data loss. The action belongs in the same design discussion as the cardinality, not in a later cleanup pass.

Several directions remain for further work. The schema could be analysed for Boyce-Codd normal form and decomposed where dependencies warrant it. Indexing strategy for foreign key attributes deserves examination, since InnoDB creates an index for each foreign key automatically and the interaction with query plans is not always what the designer expects.

Multi-tuple business rules that key constraints cannot express could be implemented with triggers, and the difference in guarantee between a trigger and a declarative constraint under concurrency is itself worth measuring. Finally, repeating the control experiment against a document database, where the equivalent guarantees are generally not available, would sharpen the comparison between the relational model and the alternatives surveyed in Module 1.

REFERENCES

[1] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed., McGraw-Hill Education, 2019.

[2] E. F. Codd, 'A Relational Model of Data for Large Shared Data Banks', Communications of the ACM, vol. 13, no. 6, pp. 377-387, 1970.

[3] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed., Pearson, 2016.

[4] C. J. Date, An Introduction to Database Systems, 8th ed., Pearson, 2003.

[5] Oracle Corporation, MySQL 8.0 Reference Manual, Section 15.1.20.5, 'FOREIGN KEY Constraints'. Available at dev.mysql.com/doc.

[6] Oracle Corporation, MySQL 8.0 Reference Manual, Section 13.3.1, 'CHECK Constraints'. Available at dev.mysql.com/doc.